00 01 02 **03** 04 05 06 07 08 09 0A 0B 0C 0D

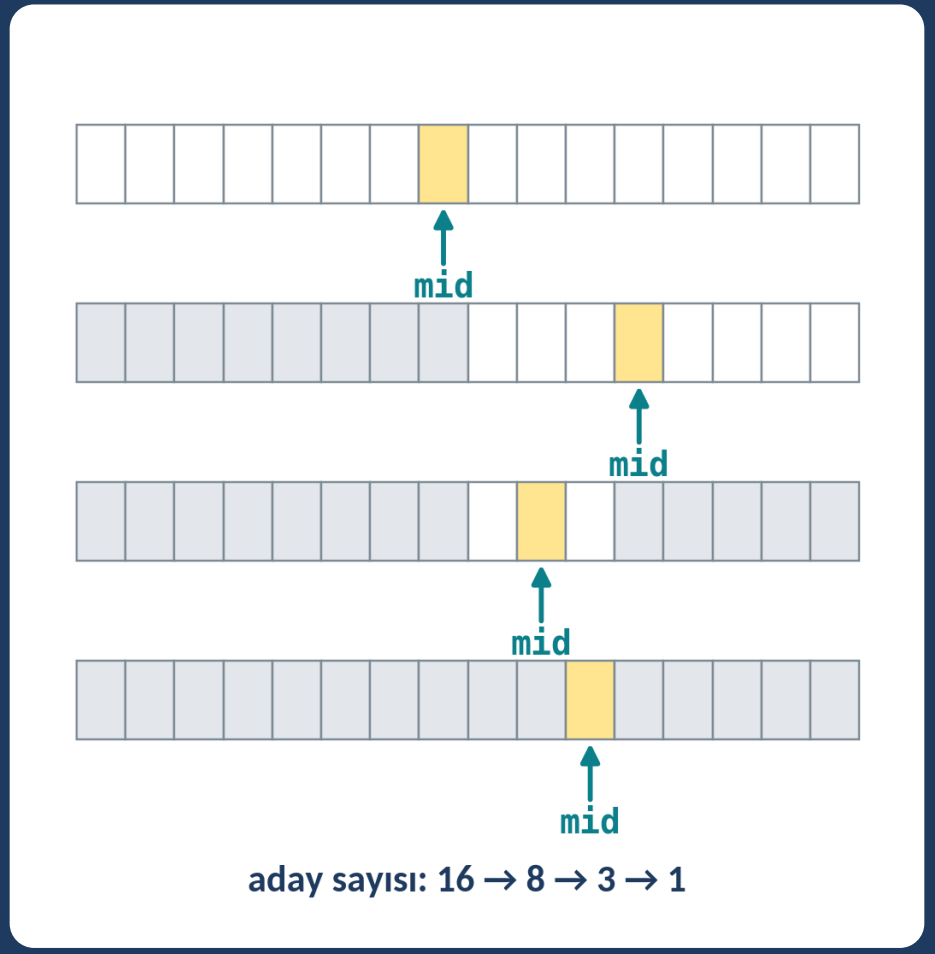
ALGORİTMA VE PROGRAMLAMA – II

HAFTA 04

Arama Algoritmaları

Doğrusal Arama, İkili Arama ve Varyantları

MODÜL II · ARAMA VE SIRALAMA



Dr. Öğr. Üyesi Mehmet Sevri · RTEÜ Bilgisayar Mühendisliği · 2026–2027 Bahar

14 HAFTALIK YOLCULUK

Yol haritasında neredeyiz?

H1 Doğruluk ve Ölçüm	**H2** Asimptotik Analiz	**H3** Özyineleme ve Geri İzleme	**H4** **Arama Algoritmaları**	**H5** Temel Sıralama	**H6** Böl ve Fethet	**H7** Öbek ve Doğrusal Sıralama
0x00	0x01	0x02	0x03	0x04	0x05	0x06

H8 Dinamik Dizi ve Bağlı Liste	**H9** Yığın ve Kuyruk	**H10** Karma Tabloları	**H11** Graflar: BFS ve DFS	**H12** Açgözlü Algoritmalar	**H13** Dinamik Programlama	**H14** Sentez ve Final Projesi
0x07	0x08	0x09	0x0A	0x0B	0x0C	0x0D

Vize: H1–H7 · Ara proje: H7 (teslim: H8 başı) · Final projesi: H14

TAHMİN ET

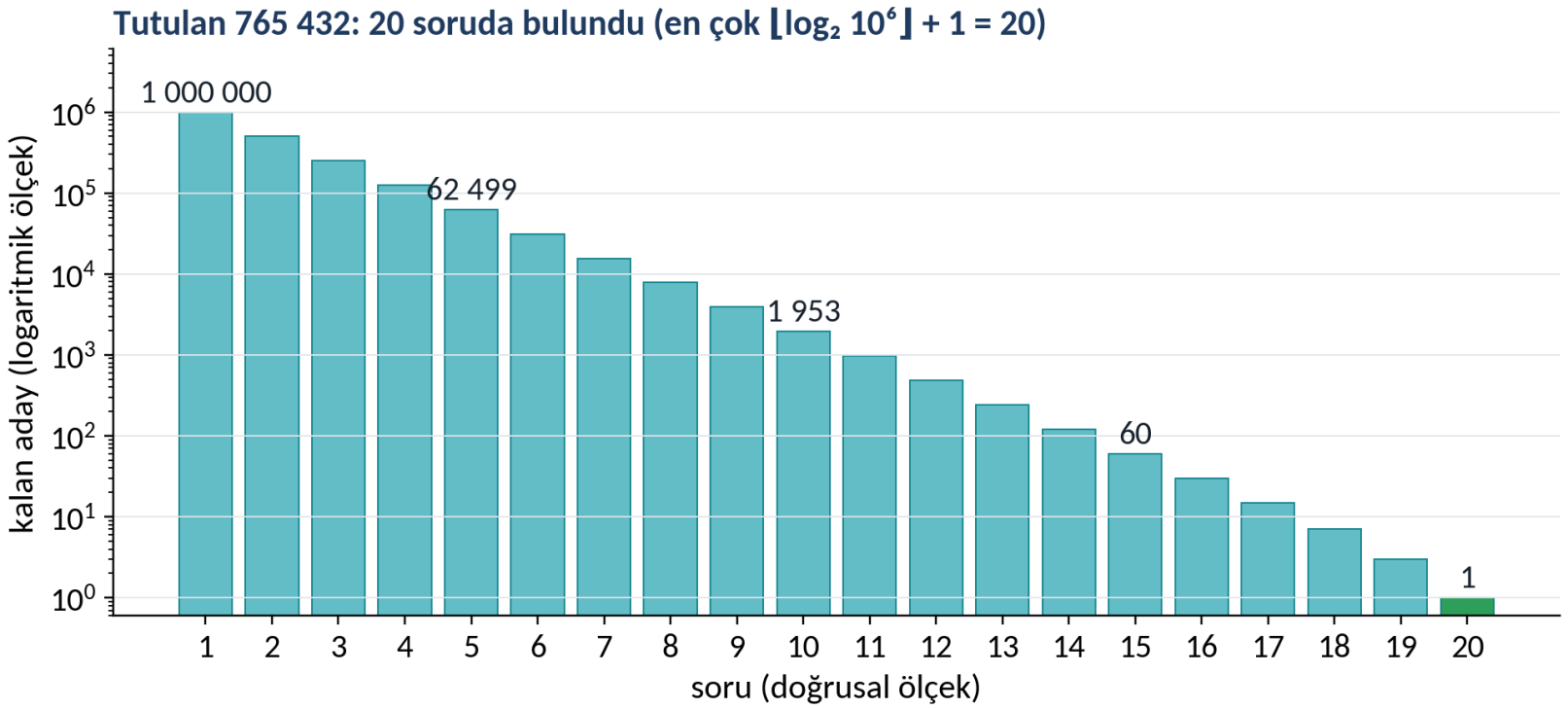
Geçen haftalardan üç soru

- **S1 (H2).** n = 100'den başlayıp her adımda `n = n / 2` yapan döngü n = 0 olana kadar kaç adım sürer?
- **S2 (H3).** T(n) = T(n/2) + c bağıntısını yerine koyma yöntemiyle çözün.
- **S3 (H1).** Doğrusal aramanın değişmezi “x, a[0..i − 1] içinde yok” ise döngü i = n'de bitince ne söyleyebiliriz?

Yanıtlar: üç araç, tek algoritma

- **S1:** 7 adım. H2'deki izleme: `İzleme (n = 100): 100 -> 50 -> 25 -> 12 -> 6 -> 3 -> 1 -> 0   (7 adım)` → $\lfloor \log_2 n \rfloor + 1$.
- **S2:** $T(n) = T(n/2^k) + k \cdot c$; $n/2^k = 1$ için $k = \log_2 n$ → **$\Theta(\log n)$**.
- **S3:** Değişmez bütün diziyi kapsar: x dizide yoktur, −1 döndürmek doğrudur.
- Bugün ikili arama bu üçünü birlikte kullanır: **yarıya inen döngü** (kaç adım), **yineleme bağıntısı** (özyinelemeli sürüm), **değişmez** (neden doğru).

Büyük Soru: bir milyon isimde kaç soru?



- Her soru adayların **yarısını** eler.
- $2^{20} = 1\,048\,576 > 10^6$: **20 soru** yeter.
- Tutulan 765 432: 20 soruda (Örnek 10).
- 10^6 elemanda 1000 sorgu: hiçbiri 20 yoklamayı aşmadı (Örnek 3).
- Peki neden yıllarca **hatalı** yazıldı?

GERÇEK DÜNYADAN

1946 · 1962 · 2006

“Neredeyse bütün ikili aramalar hatalı”

- Joshua Bloch, 2006: Java kütüphanesindeki `int mid = (low + high) / 2;` satırı low + high $2^{31} - 1$'i aşınca **negatife taşar**; Java'da `ArrayIndexOutOfBoundsException`, C'de öngörülemeyen dizi dışı erişim.
- Bloch'a göre 2^{30} (≈ 1 milyar) ve üstü elemanlı dizilerde çıkar (bizim programda ilk taşma $n = 2^{30} + 1$); JDK'de yaklaşık **dokuz yıl** fark edilmedi. Bentley'nin kanıtlayıp sınadığı ikili aramanın karşılık gelen satırı da aynı taşmayı taşıyordu.
- Düzeltme: `low + ((high - low) / 2)` ya da Java'da `(low + high) >>> 1`. Bentley'nin Knuth'a dayanarak aktardığına göre ilk ikili arama 1946'da yayımlandı, her n için doğru ilk sürüm 1962'ye kadar görülmedi (Bentley, 2000).

Bloch, J. (2006, June 2). Extra, extra – read all about it: Nearly all binary searches and mergesorts are broken. Google Research Blog.

ÖĞRENME ÇIKTILARI

Bu hafta sonunda neler yapabileceksiniz?

ÖÇ 4.1	Doğrusal aramayı (nöbetçili sürüm dahil) gerçekler; en iyi, en kötü ve ortalama durum karmaşıklığını çıkarır.
ÖÇ 4.2	İkili aramayı yarı açık aralık değişmeziyle yinelemeli ve özyinelemeli olarak doğru biçimde gerçekler; O(log n) karmaşıklığını gerekçelendirir.
ÖÇ 4.3	İkili arama varyantlarını (ilk/son konum, lower_bound/upper_bound, ekleme konumu, döndürülmüş dizide arama) uygular.
ÖÇ 4.4	Monoton bir yüklem üzerinde “cevap üzerinde ikili arama” ile optimizasyon problemlerini çözer.
ÖÇ 4.5	Fonksiyon işaretçileri ve karşılaştırıcılarla genel (generic) arama fonksiyonu yazar; bsearch, std::lower_bound ve bisect karşılıklarını kullanır.

Arama bir sözleşmedir: üç soru

a[0..n − 1] ve x verildiğinde a[k] = x olan bir k'yi ya da “bulunamadı”yı döndür.

Bulunamadı nasıl bildirilir?

C/C++: **−1** (geçersiz indis). Python: **None**; `list.index` ise **ValueError** fırlatır.

Tekrar eden değerde hangi konum?

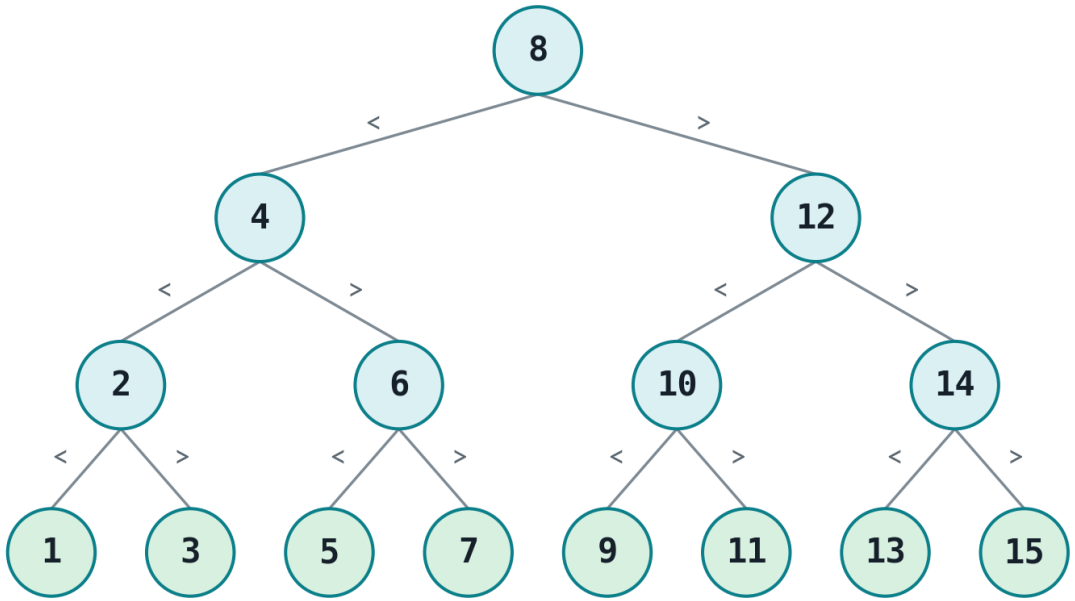
Doğrusal: **ilk** konum. Kapalı aralıklı ikili: eşitlerden **herhangi biri**. lower_bound: **ilk**. `bsearch`: belirtilmemiş.

Ön koşul ne?

Doğrusal: yok. İkili: dizi **artan sıralı**. Bozulursa uyarı yok, yalnızca yanlış sonuç (Hata Avcısı 4).

Karar ağacı: neden $\lfloor \log_2 n \rfloor + 1$ soru hem yeter hem gerekir?

1..15 arasında tutulan sayı: her soru bir düğüm



1. soru: 1 düğüm
2. soru: 2 düğüm
3. soru: 4 düğüm
4. soru: 8 düğüm

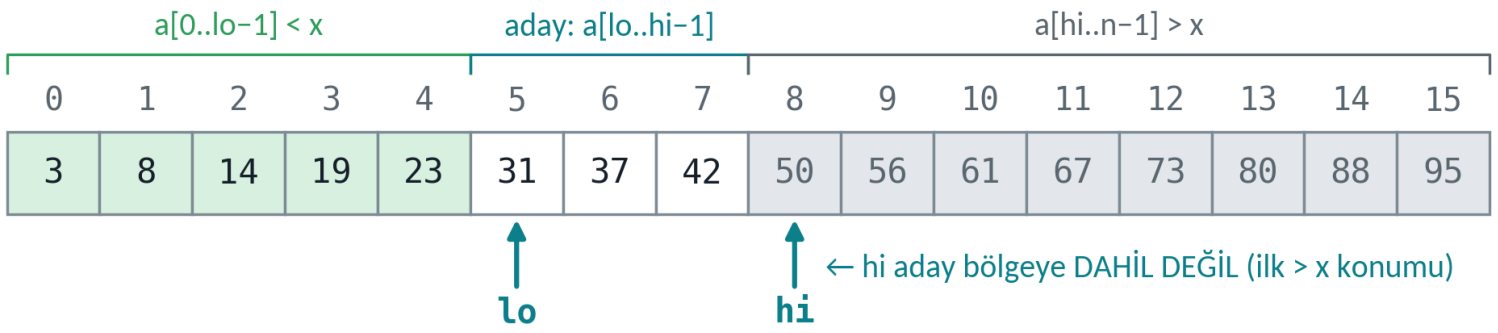
15 sayı → en çok 4 soru = $\lfloor \log_2 15 \rfloor + 1$

N	en çok
10	4
10^2	7
10^3	10
10^4	14
10^5	17
10^6	**20**
10^7	24
10^9	30

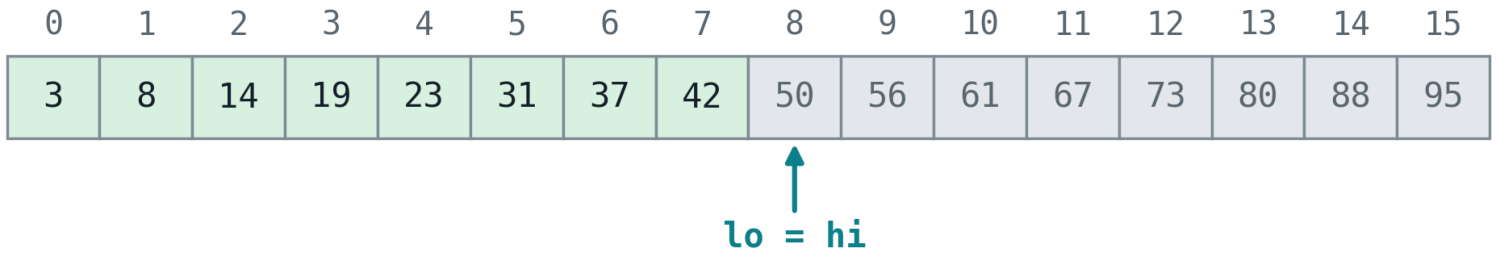
- Her düğüm bir soru: “x bu mu? değilse küçük mü, büyük mü?”
- k düzeyde en çok **$2^k - 1$** düğüm → $2^k - 1 \geq n$ gerekir.
- En küçük k = $\lceil \log_2(n + 1) \rceil$ = **$\lfloor \log_2 n \rfloor + 1$**.
- n = 10^6: 20 soru; n = 10^9: 30 soru.

Aralık sözleşmesi: dört öge birbirine uymalı

Yarı açık aralık [lo, hi): x = 45, lo = 5, hi = 8



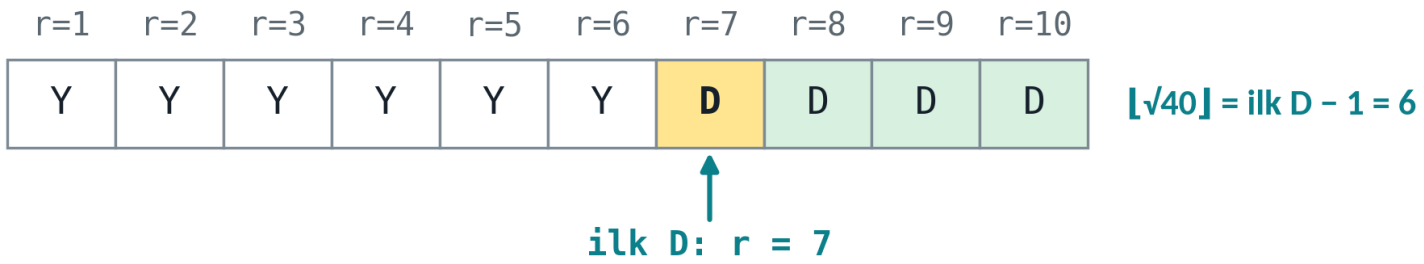
Döngü sonu: lo == hi == 8; aday bölge boş → x yok; 8 ekleme konumudur



- **Kapalı:** hi = n − 1, `lo <= hi`, hi = mid − 1
- **Yarı açık:** hi = n, `lo < hi`, **hi = mid**
- İkisinde de lo = mid + 1.
- Aday sayısı her adımda **kesin** azalır.
- Sonda lo == hi: **ekleme konumu**.

Monoton yüklem: aradığımız “ilk D”

Monoton yüklem P(r) = r·r > 40: önce hep Y (yanlış), sonra hep D (doğru)



Aynı kalıp: Y…Y D…D şeridinde ilk D'yi ikili aramayla bul

lower_bound: a[i] ≥ x

karekök: r·r > n

kapasite c yeter mi?

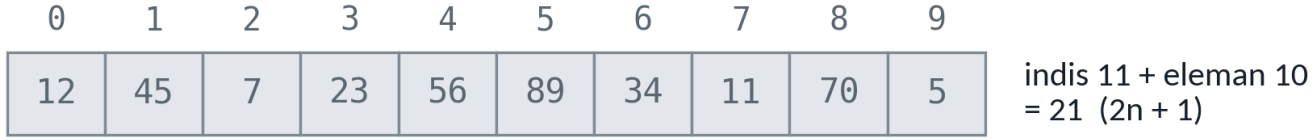
sürüm kötü mü? (git bisect)

- Y Y … Y D D … D: bir kez doğru olunca hep doğru.
- lower_bound: P(i) = “a[i] ≥ x”.
- Karekök: P(r) = “r·r > 40” → ilk D r = 7, ⌊√40⌋ = 6.
- Kapasite, ikiye bölme, git bisect: aynı kalıp.

Doğrusal arama ve nöbetçi: 2n + 1 yerine n + 2 karşılaştırma

x = 99 dizide yok (n = 10): karşılaştırma sayısı

standart: her adımda i < n ve a[i] == x



nöbetçili: a[n] = x yazılır, döngüde yalnız a[i] != x



- Sırasız veride tek seçenek: “yok” demeden önce n elemanın hepsi.
- Klasik döngü her adımda iki soru sorar: `i < n` ve `a[i] == x`.
- **Nöbetçi:** x'i a[n]'ye yaz; döngü mutlaka durur, sınır denetimi bir kez.
- n = 10, x = 99: `dogrusal_ara -> yok indis 11 + eleman 10 = 21`; `nobetcili_ara -> yok indis 1 + eleman 11 = 12`

Doğrusal ve nöbetçili arama: sözde kod ve C

```
algoritma DOĞRUSAL-ARA(a[0..n−1], x)
  i ← 0'dan n−1'e kadar her i için
    ▷ x, a[0..i−1] içinde yok
    eğer a[i] = x ise döndür i
  döndür −1
algoritma NÖBETÇİLİ-ARA(a[0..n], x)
  eski ← a[n];  a[n] ← x;  i ← 0
  a[i] ≠ x iken i ← i + 1
  a[n] ← eski
  eğer i < n ise döndür i
  döndür −1
```

c/H04_02_nobetcili_arama.c · satır 36–49 — C

```c
static long nobetcili_ara(int a[], size_t n, int x)
{
    int eski = a[n];
    a[n] = x;                /* nöbetçi */
    size_t i = 0;
    eleman_kars++;
    while (a[i] != x) {
        i++;
        eleman_kars++;
    }
    a[n] = eski;             /* geri yükle */
    indis_kars++;            /* tek sınır denetimi */
    return i < n ? (long)i : -1;
}
```

Θ(n), O(1) bellek. Örnek 2, başarısız arama: 2n + 1 → n + 2 karşılaştırma (n = 10: 21 → 12; n = 10^6: 2 000 001 → 1 000 002).

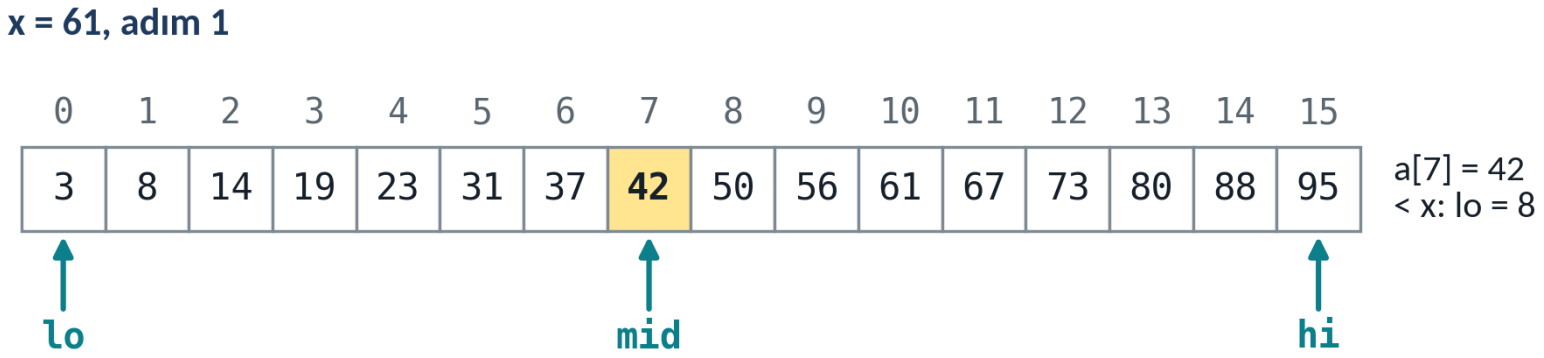
ALGORİTMA KİMLİK KARTI

Doğrusal arama *linear search; sentinel variant*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Kaba kuvvet (tek geçiş; erken çıkış; nöbetçi tekniği)	Yok — dizi sıralı olmak zorunda değil (sıralıysa a[i] > x görülünce erken çıkılabilir)	O(1)	Θ(n)	Θ(n)	O(1)

Özellikler	Başarılı aramada ortalama (n + 1)/2, başarısızda n karşılaştırma (n = 1000: 500,50 / 1000,00). Nöbetçi (x, a[n]'ye yazılır) sınır denetimini döngüden çıkarır: başarısız aramada toplam karşılaştırma 2n + 1'den n + 2'ye iner (n = 10^6: 2 000 001 → 1 000 002); dizinin sonunda yazılabilir bir boş yer gerekir
Değişmez / doğruluk	Döngü başında x, a[0..i−1] içinde yok
Kütüphane karşılığı	C: yok (elle; POSIX `lfind`) · C++: `std::find` (bulamazsa `end()`) · Python: `x in a`, `a.index(x)` (bulamazsa `ValueError`)
Ne zaman kullanılır	Veri sırasızsa; tek seferlik aramada; küçük dizilerde (bu makinede ikili arama C -O2'de n ≈ 43, C++ -O2'de n ≈ 59 sonrasında öne geçti; Python'da küçük n'de başa baş); yalnızca sıralı erişilen yapılarda (bağlı liste, dosya akışı)
Ne zaman kullanılmaz	Büyük, sıralı veride çok sayıda sorgu yapılacaksa (ikili arama; anahtar–değer için karma tablosu, H10)

İkili arama: ortadaki elemana bak, yarısını at (x = 61)

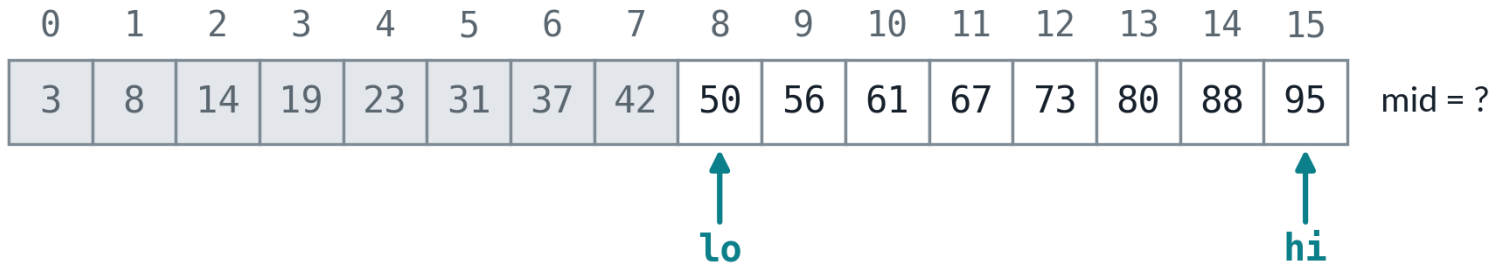


- Sıralı a[0..15]; aday bölge [lo, hi] = [0, 15]. mid = 0 + (15 − 0)/2 = **7**.
- a[7] = 42: < x: lo = 8 → a[0..7]'nin hiçbiri 61 olamaz.
- Rehberde “Kaya”yı ararken ortadan açıp “M”yi görünce M'den sonrasını atmak gibi.

TAHMİN ET · SİZCE SIRADA NE OLACAK?

lo = 8, hi = 15: bir sonraki adımda hangi eleman yoklanır?

x = 61, adım 2: lo = 8, hi = 15



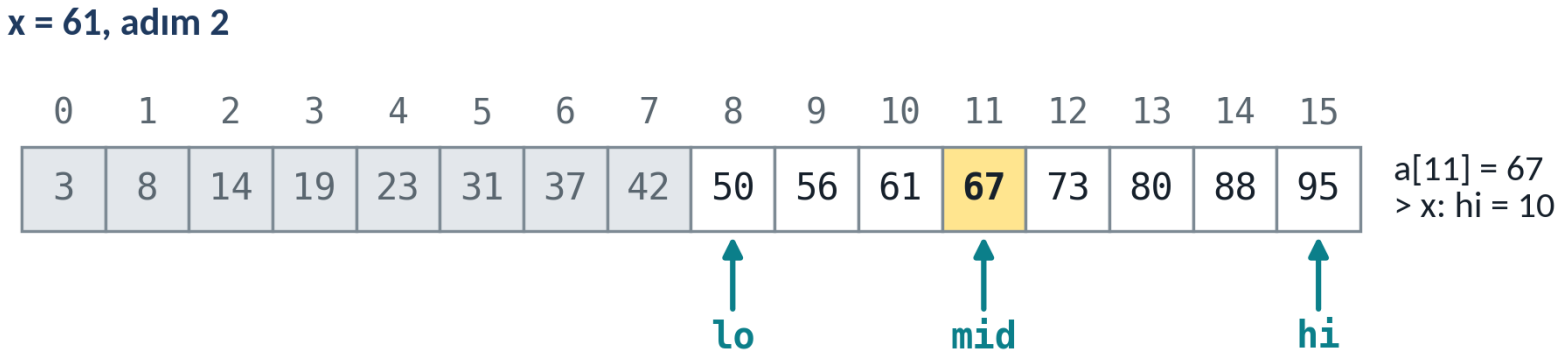
A a[12] = 73

B a[11] = 67

C a[8] = 50

D a[15] = 95

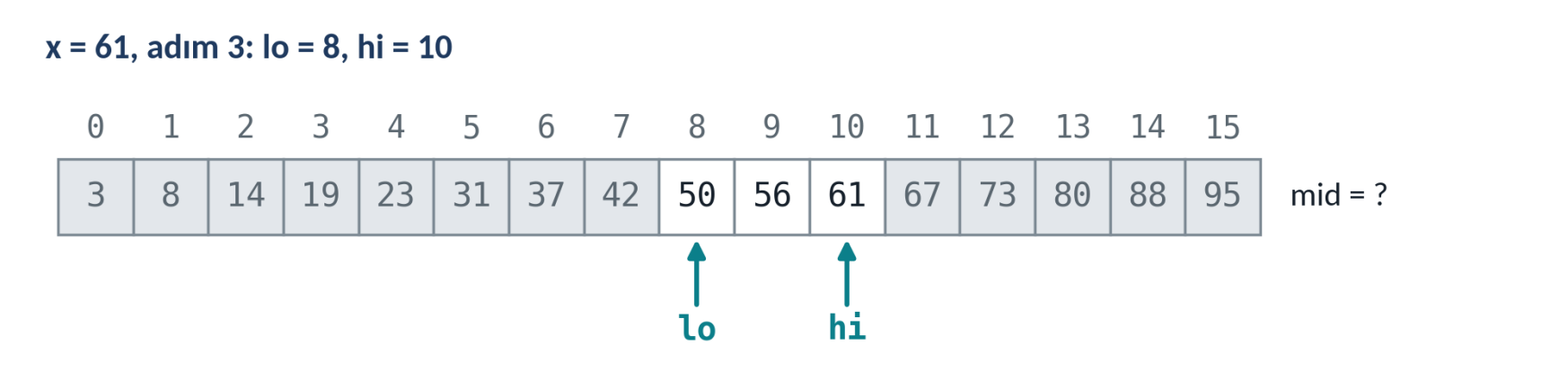
Adım 2: a[11] = 67 > 61 → sağ yarı elenir



- a[11] = 67: > x: hi = 10. Kalan aday bölge [8, 10]: yalnızca 3 aday.
- Her adımda **mid** de bölgeden çıkarılır: lo = mid + 1 ya da hi = mid − 1.

TAHMİN ET · SİZCE SIRADA NE OLACAK?

lo = 8, hi = 10: hangi eleman yoklanır, karar ne olur?



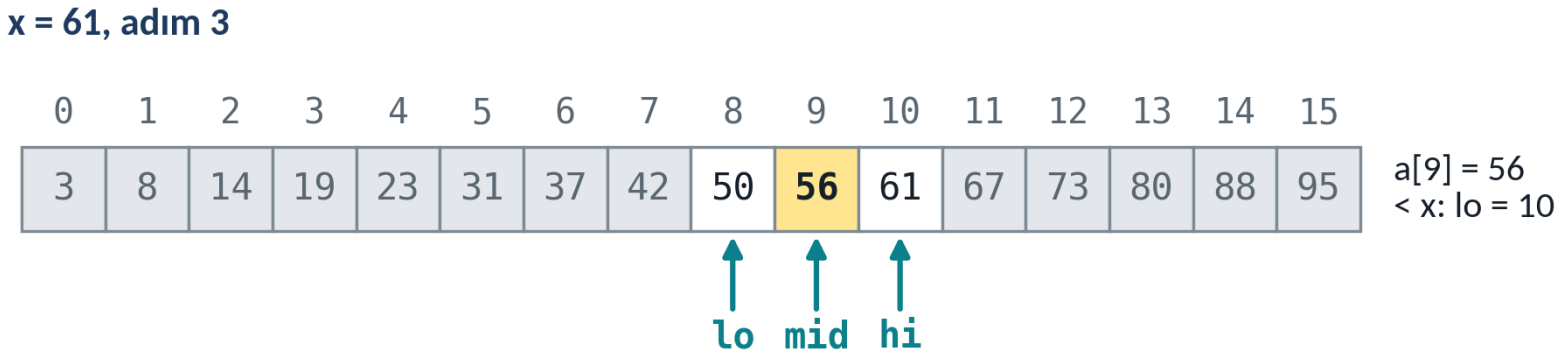
A a[10] = 61: bulundu

B a[8] = 50: lo = 9

C a[11] = 67: hi = 10

D a[9] = 56: lo = 10

Adım 3: a[9] = 56 < 61 → lo = 10

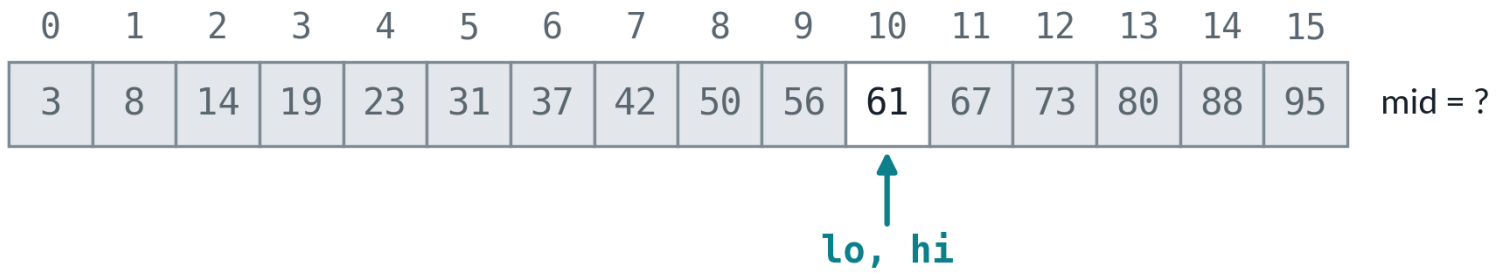


- a[9] = 56: < x: lo = 10. Aday bölge [10, 10]: **tek aday** kaldı.
- Değişmez: “61 dizideyse a[lo..hi] içindedir” her adımda korunuyor.

TAHMİN ET · SİZCE SIRADA NE OLACAK?

lo = hi = 10: döngü devam eder mi, sonuç ne olur?

x = 61, adım 4: lo = 10, hi = 10



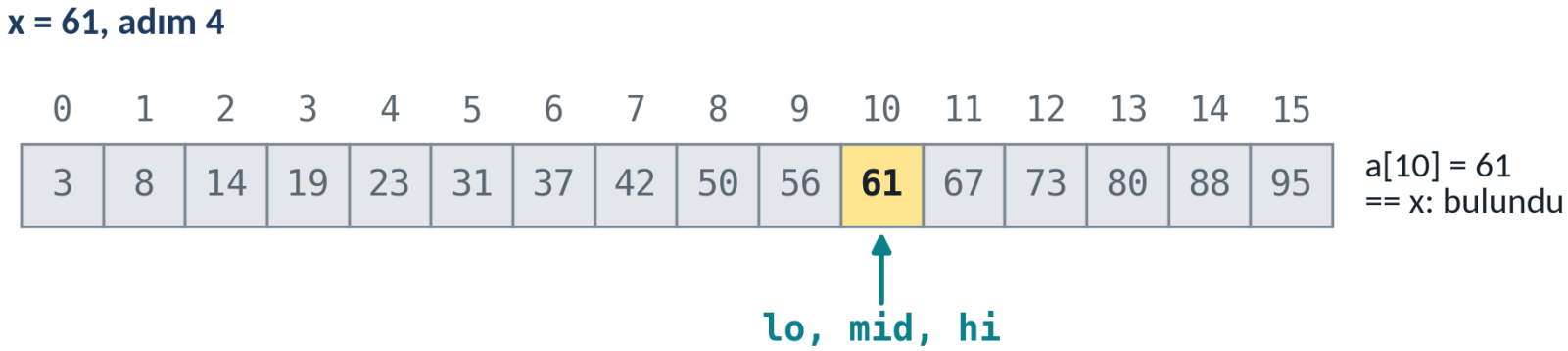
A Devam eder: a[10] = 61, bulundu (konum 10)

B Biter: lo == hi, −1 döner

C Biter: konum 10 yoklanmadan döndürülür

D Devam eder: lo = 11, sonra −1

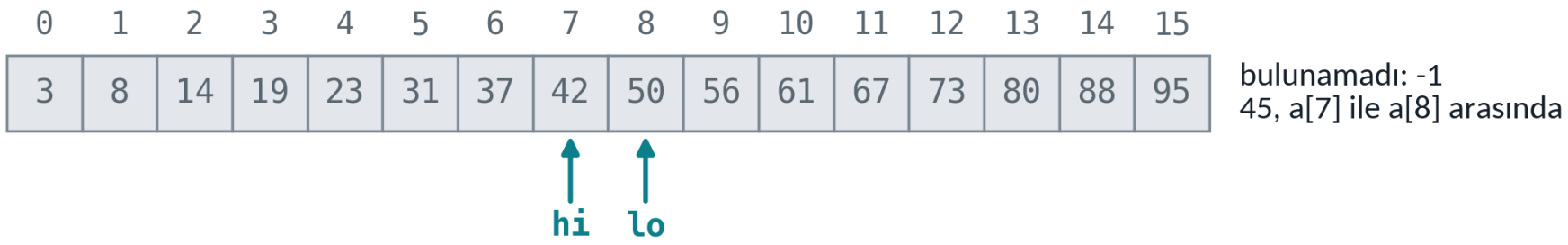
Adım 4: bulundu, 4 yoklamada



- a[10] = 61 = x → bulundu: `ikili_ara(61) = 10 yoklama = 4`.
- 16 elemanda en çok $\lfloor \log_2 16 \rfloor + 1 = \mathbf{5}$ yoklama; 95 için 5 gerekti.
- Doğrusal arama 61'i 11 karşılaştırmada bulurdu.

Bulunamayan x = 45: aralık boşalır, lo ekleme konumudur

x = 45: döngü bitti, lo = 8 > hi = 7 (aday bölge boş)



adım	lo	hi	mid	a[mid]	karar
1	0	15	7	42	< x: lo = 8
2	8	15	11	67	> x: hi = 10
3	8	10	9	56	> x: hi = 8
4	8	8	8	50	> x: hi = 7

İkili arama: sözde kod ve C (kapalı aralık)

```
algoritma İKİLİ-ARA(a[0..n-1], x)
  lo ← 0;  hi ← n − 1
  lo ≤ hi iken   ▷ aday: a[lo..hi]
    mid ← lo + ⌊(hi − lo) / 2⌋
    eğer a[mid] = x ise döndür mid
    eğer a[mid] < x ise lo ← mid + 1
    değilse hi ← mid − 1
  döndür −1      ▷ lo = hi + 1
```

c/H04_03_ikili_arama.c · satır 23–33 — C

```c
    while (lo <= hi) {
        long mid = lo + (hi - lo) / 2; /* taşmasız */
        yoklama++;
        if (a[mid] == x) {
            return mid;
        } else if (a[mid] < x) {
            lo = mid + 1;   /* a[lo..mid] elendi */
        } else {
            hi = mid - 1;   /* a[mid..hi] elendi */
        }
    }
```

22: `long lo = 0, hi = (long)n - 1;` · 34: `return -1;` · başlatma, koruma, sonlanma · **Θ(log n)**, O(1).

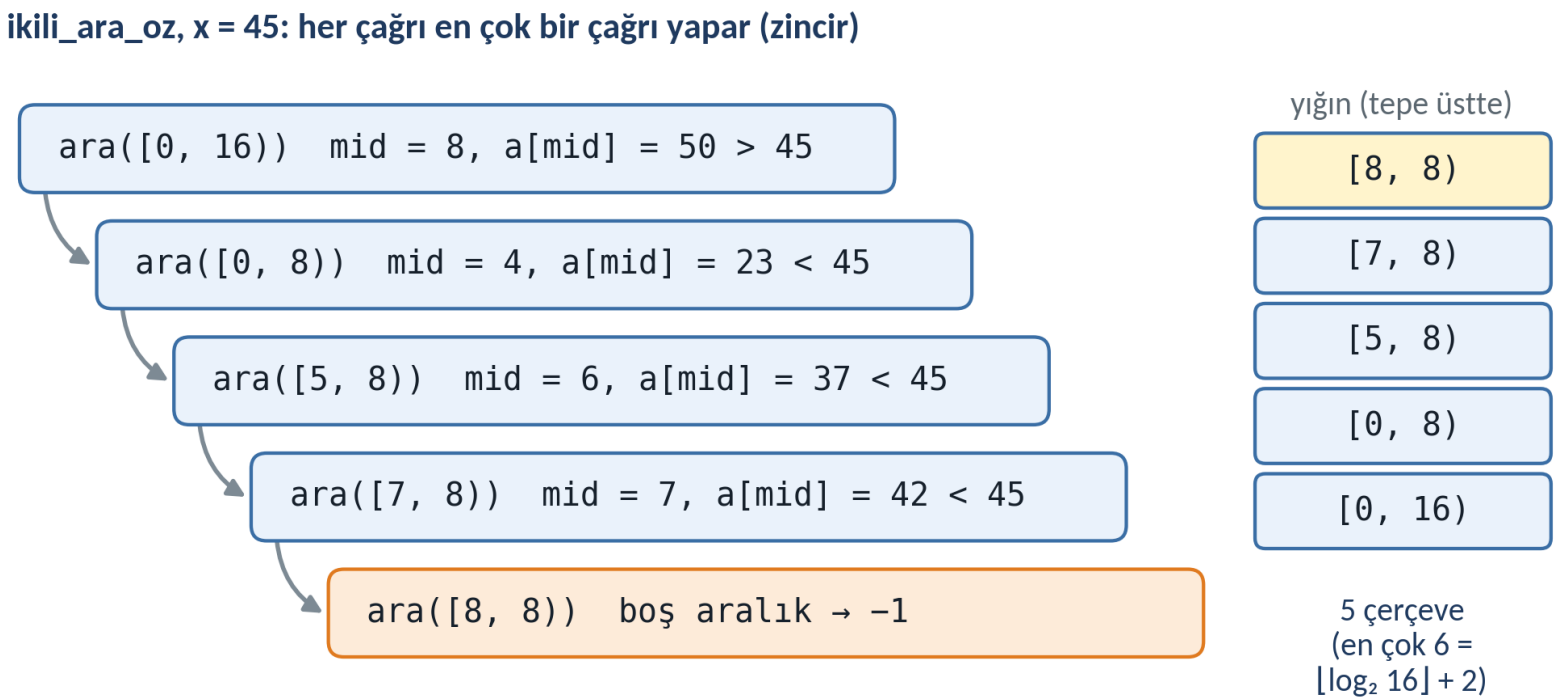
ALGORİTMA KİMLİK KARTI

İkili arama *binary search*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Azalt ve fethet (her adımda aday aralık yarıya iner)	Dizi artan sıralı ve rastgele erişimli (a[i]'ye O(1) erişim)	**O(1)**	**Θ(log n)**	**Θ(log n)**	**O(1) yinelemeli / O(log n) özyinelemeli**

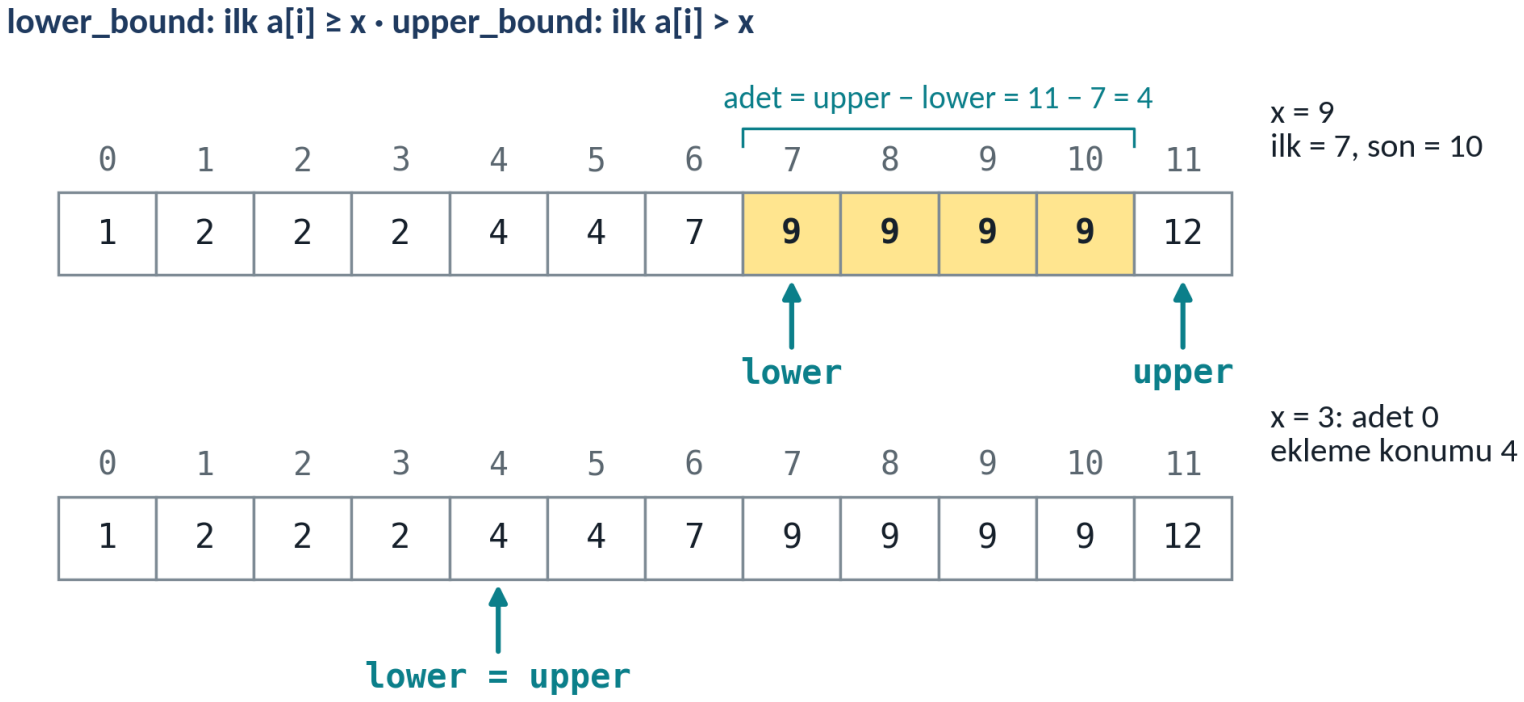
Özellikler	En çok $\lfloor \log_2 n \rfloor + 1$ yoklama ($n = 10^6$: 20); $T(n) = T(n/2) + c$. Aralık sözleşmesi tek yerde seçilir: kapalı [lo, hi] (hi = n − 1, lo <= hi, hi = mid − 1) ya da yarı açık [lo, hi) (hi = n, lo < hi, hi = mid). Orta nokta taşmasız: lo + (hi − lo) / 2
Değişmez / doğruluk	x dizideyse a[lo..hi] içindedir; a[0..lo−1] < x ve a[hi+1..n−1] > x
Kütüphane karşılığı	C: `bsearch` (`void *` taban + karşılaştırıcı) · C++: `std::binary_search` (var/yok), `std::lower_bound` · Python: `bisect.bisect_left` + eşitlik denetimi
Ne zaman kullanılır	Sıralı, rastgele erişimli veride çok sayıda arama; n büyükken (bu makinede C -O2'de n ≈ 43 sonrasında doğrusal aramadan hızlı; Python'da küçük n'de başa baş)
Ne zaman kullanılmaz	Veri sıralı değilse (önce sıralama: H5–H7) ya da sık ekleme/silme varsa (her ekleme O(n) kaydırma; dengeli arama ağaçları Veri Yapıları dersinde); çok küçük dizilerde

Özyinelemeli ikili arama: T(n) = T(n/2) + c



- Aday bölge parametre olarak taşınır: ARA(a, lo, hi, x).
- Her çağrı **en çok bir** çağrı yapar: ağaç bir zincir.
- Derinlik en çok $\lfloor \log_2 n \rfloor + 2$ (n = 4096: 14 çağrı) → yığın O(log n).
- GCC -O2 kuyruk çağrısını döngüye çevirdi; güvence değil.

lower_bound ve upper_bound: eşitlikte durmayan arama



- **lower_bound:** ilk a[i] ≥ x. **upper_bound:** ilk a[i] > x.
- x = 9: 7 ve 11 → adet 11 − 7 = 4; son konum 10.
- x = 3 (yok): ikisi de 4 → **ekleme konumu**.
- İki fonksiyon tek işaretle ayrılır: `<` ile `<=`.

TAHMİN ET · SİZCE SIRADA NE OLACAK?

a = [1, 2, 2, 2, 4, 4, 7, 9, 9, 9, 9, 12] dizisinde x = 4 için lower_bound ve upper_bound kaçtır?

A 4 ve 5

B 5 ve 6

C 4 ve 6

D 3 ve 6

lower_bound: sözde kod ve C (yarı açık aralık)

```
algoritma LOWER-BOUND(a[0..n-1], x)
  lo ← 0;  hi ← n
  lo < hi iken
    ▷ a[0..lo-1] < x ≤ a[hi..n-1]
    mid ← lo + ⌊(hi − lo) / 2⌋
    eğer a[mid] < x ise lo ← mid + 1
    değilse hi ← mid  ▷ sola
  döndür lo
```

c/H04_07_alt_ust_sinir.c · satır 20–33 — C

```c
static size_t alt_sinir(const int a[], size_t n, int x)
{
    size_t lo = 0, hi = n;
    while (lo < hi) {
        size_t mid = lo + (hi - lo) / 2;
        yoklama++;
        if (a[mid] < x) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }
    return lo;
}
```

`ust_sinir` (satır 37–50) yalnız bu karşılaştırmada ayrışır: `a[mid] <= x`. Erken çıkış yok: $\lfloor \log_2 n \rfloor$ ya da $\lfloor \log_2 n \rfloor + 1$ yoklama.

ALGORİTMA KİMLİK KARTI

lower_bound / upper_bound *binary search variants*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Azalt ve fethet (eşitlikte durmayan ikili arama)	Dizi artan sıralı; tekrar eden anahtarlar olabilir	Θ(log n)	Θ(log n)	Θ(log n)	O(1)

Özellikler	Erken çıkış yoktur: her arama $\lfloor \log_2 n \rfloor$ ya da $\lfloor \log_2 n \rfloor + 1$ yoklama sürer. lower_bound ilk a[i] >= x, upper_bound ilk a[i] > x konumudur; adet = upper − lower, ilk konum = lower, son konum = upper − 1; x yoksa lower ekleme konumudur (n = 10^6'da bir değerin 10 kopyası 39 yoklamada sayıldı)
Değişmez / doğruluk	lower_bound: a[0..lo−1] < x ≤ a[hi..n−1]; upper_bound: a[0..lo−1] ≤ x < a[hi..n−1]
Kütüphane karşılığı	C: yok (`bsearch` eşitlerden herhangi birini döndürür) · C++: `std::lower_bound`, `std::upper_bound`, `std::equal_range` · Python: `bisect.bisect_left`, `bisect.bisect_right`
Ne zaman kullanılır	Tekrar eden anahtarlarda konum ve sayım, aralık sorguları (lo ≤ x < hi kaç tane?), ekleme konumu, önek araması (Mini-OBS 2 v2.3)
Ne zaman kullanılmaz	Sırasız veride; yalnızca var/yok sorusunda (üç yönlü ikili arama ya da karma tablosu daha doğrudan)

Cevap üzerinde ikili arama: tamsayı karekök

```
algoritma İLK-DOĞRU(lo, hi, P)
  ▷ P monoton, P(hi) = D
  lo < hi iken
    mid ← lo + ⌊(hi − lo) / 2⌋
    eğer P(mid) ise hi ← mid
    değilse lo ← mid + 1
  döndür lo
▷ ⌊√n⌋ = İLK-DOĞRU(…) − 1
```

c/H04_10_karekok_tahmin.c · satır 36–44 — C

```c
    while (lo < hi) {
        uint64_t mid = lo + (hi - lo) / 2;
        adim++;
        if (cok_buyuk_mu(mid, n)) {
            hi = mid;       /* ilk D: mid ya da solu */
        } else {
            lo = mid + 1;   /* ilk D: mid'in sağı */
        }
    }
```

lo = 1, hi = min(n, 2^{32}); yüklem `r > n / r` (`r * r` 64 bitte taşar). n = $2^{64} - 1$: ⌊√n⌋ = 4 294 967 295, 32 adım.

TAHMİN ET · SİZCE SIRADA NE OLACAK?

Sayı tahmin oyunu (1..100): tutulan 27. İlk üç soru 50, 25, 37. Dördüncü soru hangisi?

A 30

B 31

C 32

D 26

Oyunun kendisi bir ikili arama: en çok $\lfloor \log_2 N \rfloor + 1$ soru

ÇIKTI · ÖRNEK 10, OYUN 2

```
oyun 2: tutulan 27
  soru 1: 50 mi? -> daha küçük
  soru 2: 25 mi? -> daha büyük
  soru 3: 37 mi? -> daha küçük
  soru 4: 31 mi? -> daha küçük
  soru 5: 28 mi? -> daha küçük
  soru 6: 26 mi? -> daha büyük
  soru 7: 27 mi? -> doğru
  7 soruda bulundu
```

BÜTÜN TUTULAN SAYILAR, N = 100

```
N = 100: en çok 7 soru (⌊log2 N⌋ + 1 = 7), ortalama 5.800
  soru:    1    2    3    4    5    6    7
  oyun:    1    2    4    8   16   32   37
```

- k. soruda bulunan sayılar karar ağacının k. düzeyidir: **2^{k-1}** sayı.
- N = 100: en çok 7 soru, ortalama 5,80.
- N = 10^6: 1.–19. soru düzeyleri dolu (k. soruda 2^{k-1} sayı); kalan 475 713 sayı 20. soruda bulunur; ortalama 18,95.

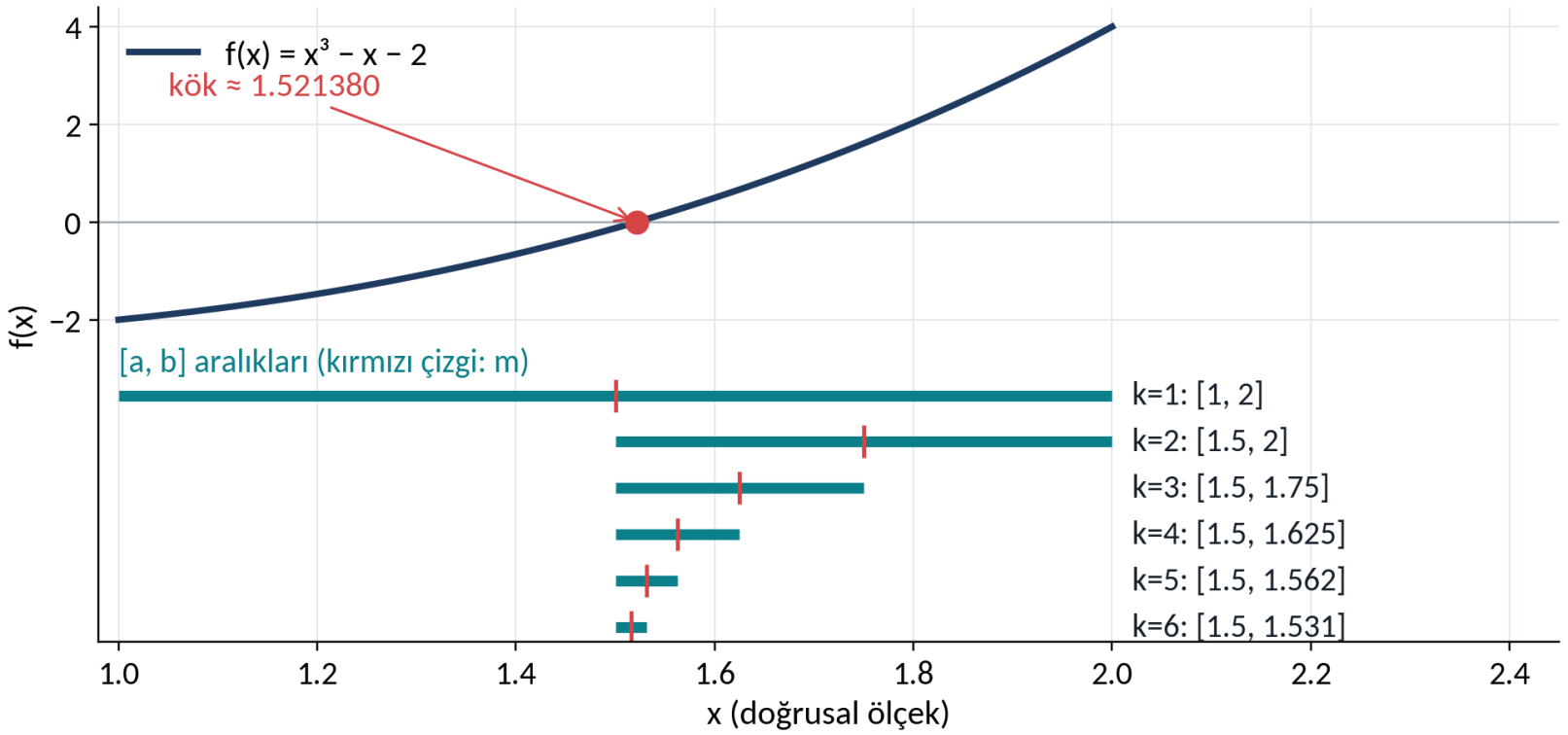
ALGORİTMA KİMLİK KARTI

Cevap üzerinde ikili arama *binary search on the answer; bisection method*

STRATEJİ	GİRDİ	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Azalt ve fethet (monoton yüklemde ilk "doğru"nun aranması)	Yanıt aralığında monoton bir yüklem: P(r) Y…Y D…D biçiminde (bir kez doğru olunca hep doğru)	—	—	O(log R · c)	O(1)

Özellikler	R: yanıt aralığının genişliği, c: yüklemin maliyeti. Tamsayı karekökte $R \le 2^{32}$: en çok 32 adım; sayı tahmin oyununda en çok $\lfloor \log_2 N \rfloor + 1$ soru ($N = 10^6$: 20); gerçel sayılarda ikiye bölme: tol toleransı için $\lceil \log_2((b-a)/tol) \rceil$ yineleme ($tol = 10^{-6}$: 20)
Değişmez / doğruluk	P(lo − 1) = Y ve P(hi) = D (ilk D [lo, hi] içindedir); ikiye bölmede f(a) ile f(b) ters işaretli
Kütüphane karşılığı	C: yok · C++: `std::partition_point` (yüklemle ikili arama) · Python: `bisect` + `key=` (3.10+); karekök için `math.isqrt`
Ne zaman kullanılır	Doğrudan çözülmesi zor ama "bu yanıt yeterli mi?" sorusu kolay ve monoton olan problemlerde (karekök, kapasite, "en büyüğün en küçüğü", denklem kökü)
Ne zaman kullanılmaz	Yüklem monoton değilse (yanlış yanıt); kayan noktada durma koşulu tam eşitlikse (sonsuz döngü: Hata Avcısı 8)

Gerçel sayılarda ikili arama: ikiye bölme yöntemi



- $f(x) = x^3 - x - 2$, [1, 2]: $f(1) < 0 < f(2)$ → kök var.
- Yüklem: f(m) ile f(a) aynı işaretli mi? → a ← m, değilse b ← m.
- $\lceil \log_2((b - a)/tol) \rceil$ yineleme: $tol = 10^{-6}$ → 20.
- $tol = 10^{-16}$: formül 54, program 52'de f(m) == 0 ile durur.

Merdiven araması: her adımda bir satır ya da bir sütun

x = 15: 4 adım, bulundu

3	6	9	3 13	2 16	1 17
5	9	12	4 15	17	20
8	12	13	17	20	24
10	13	15	18	23	28
11	15	18	21	24	30

sağ üstten başla: > x ise sola, < x ise aşağı

x = 19: 8 adım, matristen çıkıldı: yok

3	6	9	13	16	1 17
5	9	12	15	3 17	2 20
8	12	13	5 17	4 20	24
10	13	15	6 18	23	28
11	15	8 18	7 21	24	30

sağ üstten başla: > x ise sola, < x ise aşağı

- Sağ üst köşe: satırının en büyüğü, sütununun en küçüğü.
- a[r][c] > x → sütun elenir (sola); a[r][c] < x → satır elenir (aşağı).
- x = 15: 4 adım; x = 19: 8 adımda dışarı.
- En çok m + n − 1 adım: 100 × 100'de 117 (satır satır ikili 700).

Merdiven araması: sözde kod ve C

```
algoritma MERDİVEN-ARA(a, m, n, x)
  r ← 0;  c ← n − 1   ▷ sağ üst
  r < m ve c ≥ 0 iken
    ▷ x varsa a[r..m−1][0..c]
    eğer a[r][c] = x ise döndür (r, c)
    eğer a[r][c] > x ise c ← c − 1
    değilse r ← r + 1
  döndür yok
```

c/H04_14_merdiven_araması.c · satır 42–53

C

```c
    while (r < m && c > 0) {
        int v = a[r * n + c - 1];
        adim++;
        if (v == x) {
            return (long)(r * n + c - 1);
        } else if (v > x) {
            c--;            /* sütunu ele */
        } else {
            r++;            /* satırı ele */
        }
    }
    return -1;
```

Başlangıç `r = 0, c = n`: C'de `c` "sütun + 1" tutulur (`size_t` −1 olamaz), döngü `c > 0` ile denetlenir. O(m + n).

ALGORİTMA KİMLİK KARTI

Merdiven araması (2B) *staircase search; saddleback search*

STRATEJİ	GİRDİ ÖNKOŞULU	EN İYİ	ORTALAMA	EN KÖTÜ	EK BELLEK
Azalt ve fethet (her adımda bir satır ya da bir sütun elenir)	m × n matrisin satırları soldan sağa, sütunları yukarıdan aşağıya artan	O(1)	O(n + m)	O(n + m)	O(1)

Özellikler	Sağ üst köşeden başlar: a[r][c] > x ise sütun, < x ise satır elenir; en çok m + n − 1 adım (100 × 100: sınır 199, ölçülen en çok 117). Kare matriste satır satır ikili aramadan iyi (en çok 700 yoklama), çok yassı matriste kötü (4 × 1000: 1003'e karşı 40)
Değişmez / doğruluk	x matristeyse a[r..m−1][0..c] alt matrisindedir
Kütüphane karşılığı	C: yok · C++: yok (satır başına `std::lower_bound` yazılabilir) · Python: yok (satır başına `bisect`)
Ne zaman kullanılır	Satır ve sütunları sıralı tablolarda (ör. iki ölçüte göre sıralı ızgaralar) tek değer aramak
Ne zaman kullanılmaz	Matris yalnız satır bazında sıralıysa (her satırda ikili arama); satır ya da sütun sayısı diğerinden çok büyükse (kısa kenar boyunca ikili arama daha az adım sürer)

Üç dilde: lower_bound (konum türü size_t · std::size_t · int)

H04_07_alt_ust_sinir.c — C

```c
    size_t lo = 0, hi = n;
    while (lo < hi) {
        size_t mid = lo + (hi - lo) / 2;
        yoklama++;
        if (a[mid] < x) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }
    return lo;
```

H04_07_alt_ust_sinir.cpp — C++

```cpp
    std::size_t lo = 0, hi = a.size();
    while (lo < hi) {
        std::size_t mid = lo + (hi - lo) / 2;
        yoklama++;
        if (a[mid] < x) {
            lo = mid + 1;
        } else {
            hi = mid;
        }
    }
    return lo;
```

H04_07_alt_ust_sinir.py — Python

```python
    lo, hi = 0, len(a)
    while lo < hi:
        mid = (lo + hi) // 2
        yoklama += 1
        if a[mid] < x:
            lo = mid + 1
        else:
            hi = mid
```

Üç dilde: doğrusal arama (–1 · –1 · None)

H04_01_dogrusal_arama.c — C

```c
{
    for (size_t i = 0; i < n; i++) {
        karsilastirma++;
        if (a[i] == x) {
            return (long)i;
        }
    }
    return -1;
```

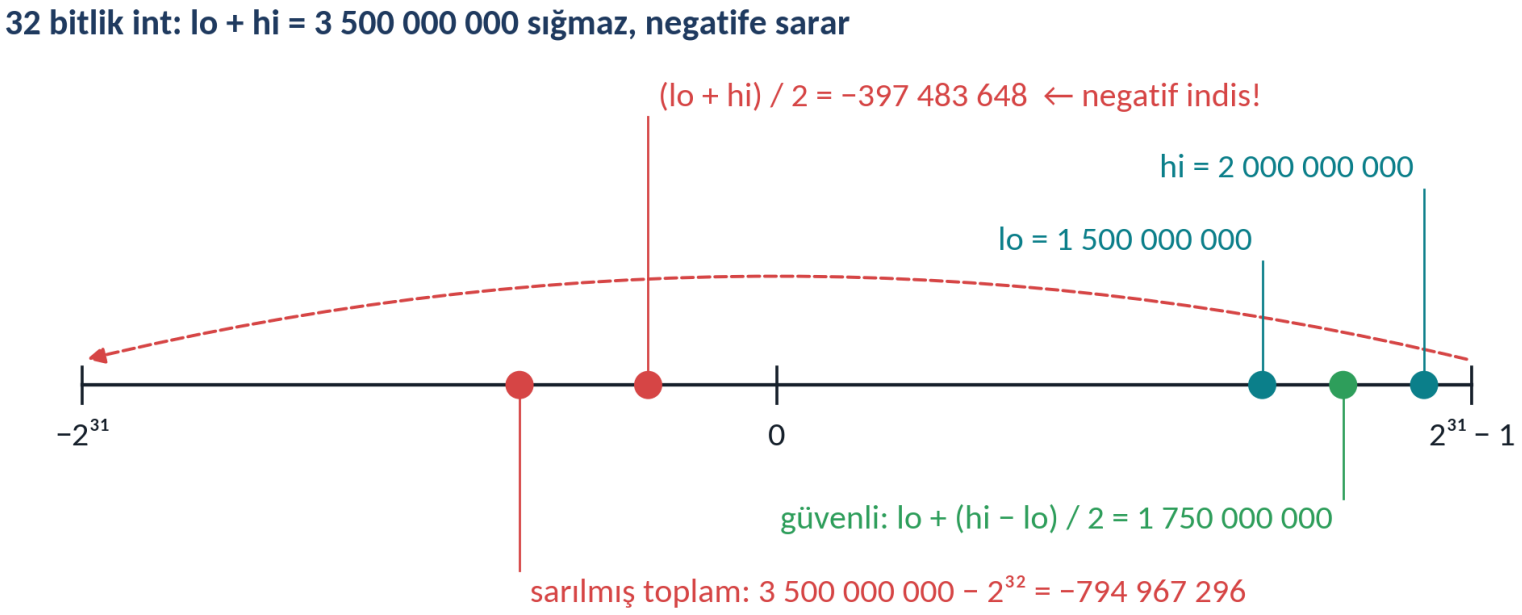
H04_01_dogrusal_arama.cpp — C++

```cpp
{
    for (std::size_t i = 0; i < a.size(); i++) {
        karsilastirma++;
        if (a[i] == x) {
            return static_cast<long>(i);
        }
    }
    return -1;
```

H04_01_dogrusal_arama.py — Python

```python
def dogrusal_ara(a, x):
    global karsilastirma
    for i in range(len(a)):
        karsilastirma += 1
        if a[i] == x:
            return i
    return None
```

Orta nokta taşması: (lo + hi) / 2 int32 sınırında

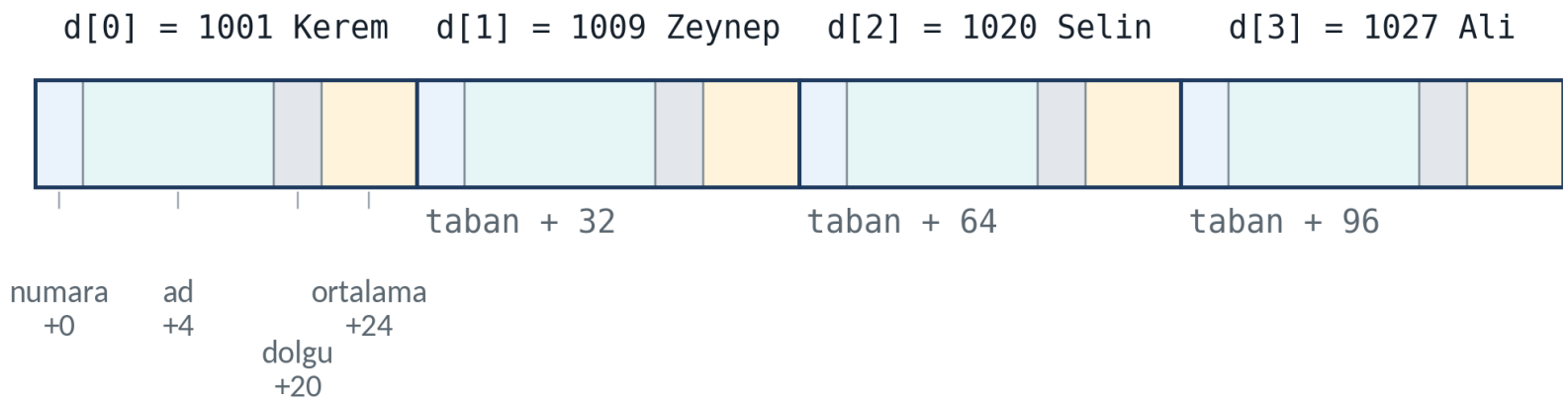


C/C++'ta işaretli taşma tanımsız davranıştır (şekil tipik sarmayı gösterir); Python'da taşma yok

- lo + hi = 3 500 000 000 > $2^{31} - 1$: sarar, ikiye bölünce **−397 483 648**.
- Güvenli: lo + (hi − lo)/2 = 1 750 000 000.
- C/C++: işaretli taşma **tanımsız davranış**; Örnek 6 bunu `uint32_t` ile benzetir.
- Hata n = $2^{30} + 1$'de başlar (bu sözleşmede). Python'da taşma yok.

Genel arama: karşılaştırıcı işaretçisi ve öge boyutu

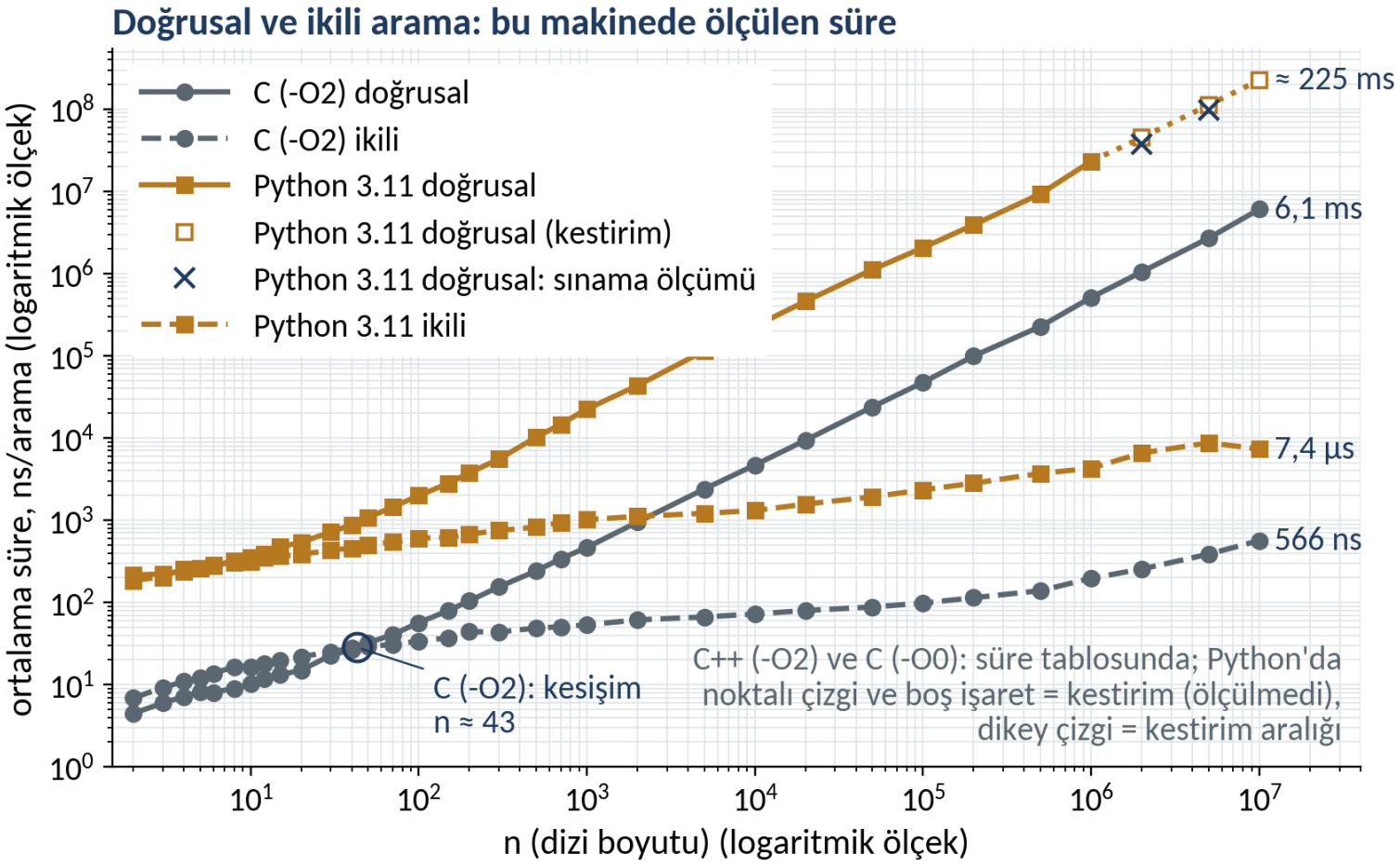
struct ogrenci dizisi: öge boyutu 32 bayt (bu makinede), i. öge taban + i × 32




```
oge = (const char *)taban + mid * boyut;   kars(anahtar, oge)
```

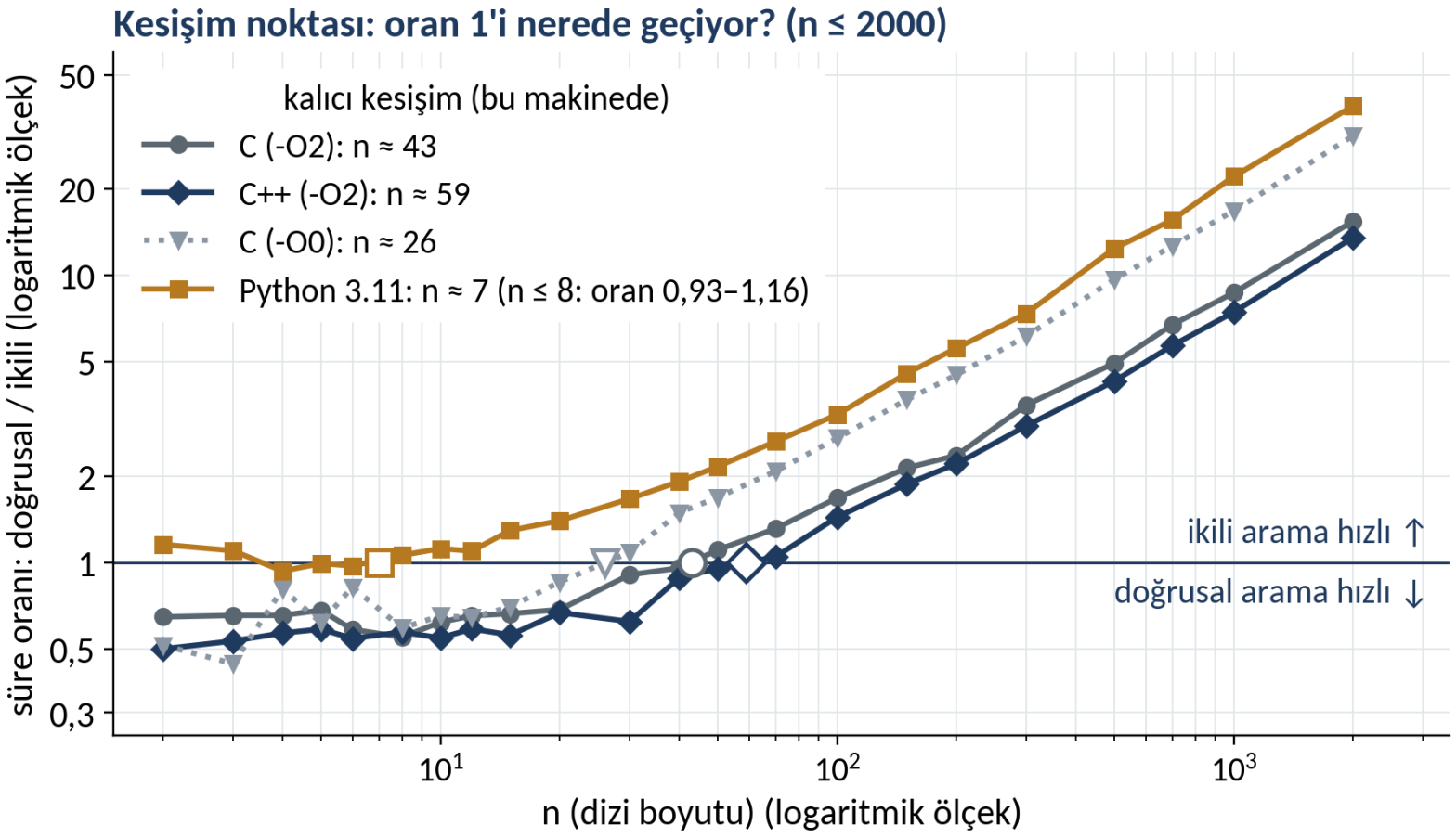
- `genel_ara(anahtar, taban, n, boyut, kars)`: `bsearch` ile aynı imza.
- mid. öge: **taban + mid × boyut** bayt (AP-1 H10).
- sizeof(struct ogrenci) = 32 bayt; dolgu 4 bayt.
- Karşılaştırıcı: `(x > y) - (x < y)`, `x - y` değil (taşma).

Haftanın Deneyi: doğrusal ve ikili arama, n = 2 … 10^7



- C -O2 kesişim: n ≈ **43** (40–50); C++ -O2: n ≈ 59.
- Python: düz doğrusal aramanın küçük-n üstünlüğü **görülmedi**; nöbetçili ≈ 14'e kadar hızlı.
- n = 10^7 ikili: C 566,3 ns, Python 7,36 µs.
- Kesişim tek sayı değil: ölçüm koşuluna bağlı bir **aralık**.

Kaputun altında: küçük dizide neden doğrusal arama kazanır?



- n = 10: doğrusal 10,2 ns, ikili 16,5 ns (C -O2).
- n = 20: doğrusal 15,2, ikili 4,1 karşılaştırma; yine doğrusal hızlı.
- Olası açıklama (ölçülmedi): ardışık erişim, **dal tahmini** hep doğru; ikilide yön rastgele.
- Büyük n'de önbellek: ikili aramanın uzak yoklamaları belleğe gider.

Hata Avcısı: dokuz hata, hiçbiri derleyici uyarısı vermez

#	Hata	Bu makinede belirti	Çözüm
1	Kapalı aralıkta lo = mid	ara(50) bitmez (zaman aşımı)	lo = mid + 1
2	hi = n ile <= / hi = n − 1 ile <	dizi dışı okuma / son eleman kaçar	tek sözleşme
3	(lo + hi) / 2	negatif mid; UBSan: overflow	lo + (hi − lo) / 2
4	sırasız diziye ikili arama	bazı elemanlar bulunamaz	önce sırala / denetle
5	karşılaştırıcıda x − y	bsearch yanlış yarıya gider	(x > y) − (x < y)
6	Python list.index	ValueError, program durur	x in a / try / None
7	son = lower_bound − 1	x = 2: son 0, adet 0	upper_bound − 1
8	while f(m) != 0.0	m değişmez, sonsuz döngü	tolerans + sınır
9	size_t ile hi = mid − 1	SIZE_MAX, SIGSEGV	yarı açık [0, n)

HA1: lo = mid aralığı daraltmaz

Hatalı: sonsuz döngü

hata_avcisi/H04_HA1_sonsuz_dongu.c · satır 34–39 — C

```c
34          if (a[mid] == x) {
35              return mid;
36          } else if (a[mid] < x) {
37              lo = mid;               /* HATA: mid zaten elendi; mid + 1 */
38          } else {
39              hi = mid - 1;
```

Düzeltilmiş

hata_avcisi/H04_HA1_sonsuz_dongu_duzeltilmis.c · satır 21–26 — C

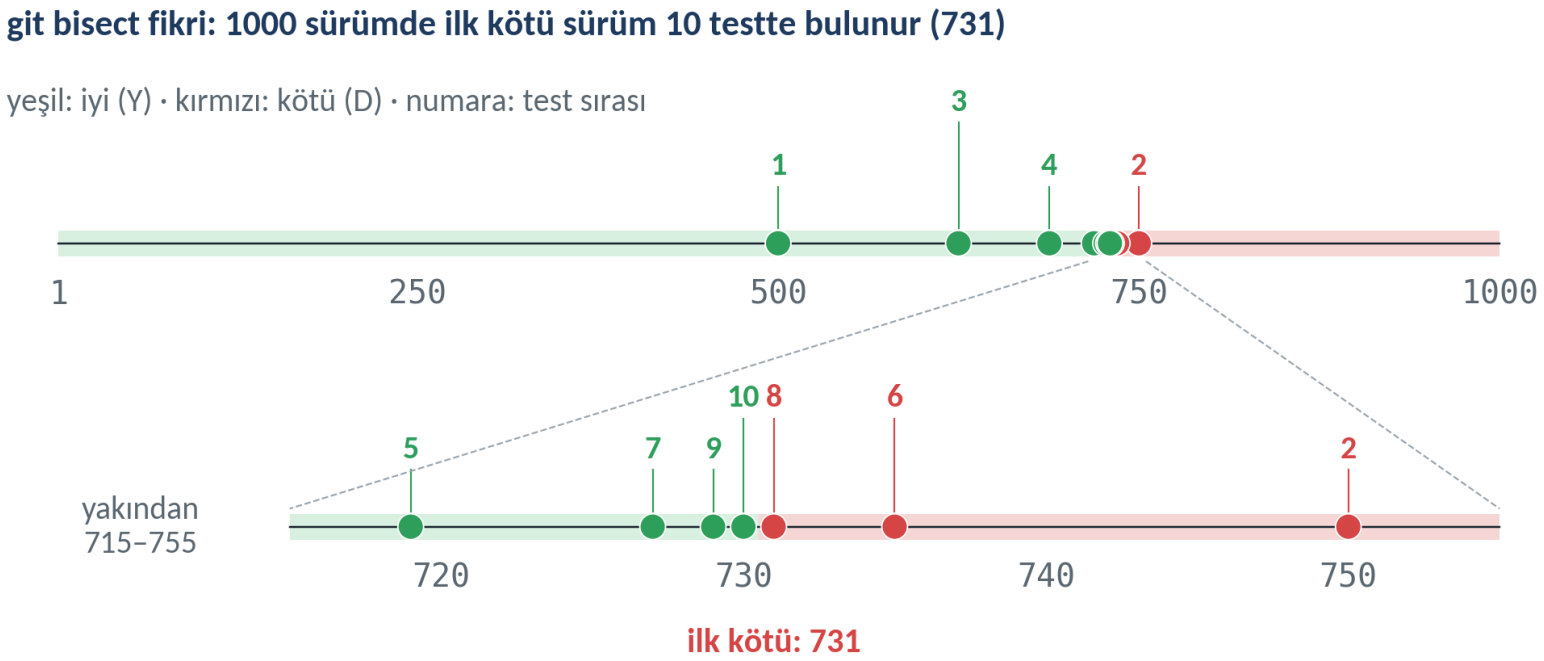
```c
21          if (a[mid] == x) {
22              return mid;
23          } else if (a[mid] < x) {
24              lo = mid + 1;           /* mid aralıktan çıkarılır */
25          } else {
26              hi = mid - 1;
```

a = [10, 20, 30, 40, 50], ara(50): lo = 3, hi = 4 iken mid = 3 ve `lo = mid` hiçbir şeyi değiştirmez. Kural: **mid her adımda bölgeden çıkarılmalı.**

Dil Penceresi: aynı algoritma, üç dilin kuralları

Öge	C	C++	Python
Bulunamadı	−1	−1; find → end()	None; index → ValueError
Orta nokta	lo + (hi − lo) / 2	aynı (C++20: std::midpoint)	(lo + hi) // 2 (taşma yok)
İşaretli taşma	tanımsız davranış	tanımsız davranış	yok
Kütüphane	bsearch (eşitlerden biri)	lower_bound, upper_bound, equal_range	bisect_left, bisect_right
Karşılaştırıcı	int (*)(const void *, const void *)	“küçüktür” (katı zayıf sıralama)	key= fonksiyonu (3.10+)
Fonksiyonu değer olarak	fonksiyon işaretçisi	şablon + lambda	birinci sınıf fonksiyon
Zincir karşılaştırma	a <= x && x < b	aynı	a <= x < b

Mühendis gözüyle: git bisect ve ön işleme ödünleşimi



- "Bu sürümde hata var mı?" zamanda monoton: iyi … iyi kötü … kötü.
- `git bisect start / bad / good v1.0` → git ortadaki sürümü çıkarır.
- k = 1000: ilk kötü sürüm 731, 10 test (⌈log2 k⌉ = 10).
- Ön işleme: 10^5 anahtarda sıralama 20 sorgudan sonra kâra geçer (Kavram 9).

MERAK KUTUSU

Fonksiyonu fonksiyona vermek; üçe bölmek neden kazandırmaz?

- `typedef double (*fonk_t)(double);` → tek `ikiye_bolme` her denklemi çözer: `x^3 - x - 2 kök ≈ 1.521379707 (30 yineleme)`
- **Üçlü arama:** adımda 2 karşılaştırma, aralık 1/3 → $2 \cdot \log_3 n \approx 1{,}26 \cdot \log_2 n > \log_2 n$. **İnterpolasyon:** karşılaştırma modelinin dışında (düzgün veride log log n).

kavram/H04_K06_fonksiyon_isaretcisi.c · satır 47–58

C

```c
    const struct {
        const char *ad;
        fonk_t f;
    } denklem[] = {{"x^3 - x - 2", kup}, {"x^2 - 2", kare}};
    for (int i = 0; i < 2; i++) {
        int k;
        double r = ikiye_bolme(denklem[i].f, 1.0, 2.0, 1e-9, &k);
        printf("%-12s kök ≈ %.9f  (%d yineleme)\n", denklem[i].ad, r, k);
    }
    fonk_t g = kare;
    printf("g = kare: g(3.0) = %.1f, (*g)(3.0) = %.1f\n", g(3.0),
           (*g)(3.0));
```

Mini-OBS 2 v2.3: numarayla ikili arama ve soyad öneki

- Kayıt dizisi her an numaraya göre **kesin artan** (değişmez); yükleme ve ekleme sıralı konuma.
- `ikili_ara_no_say` (yarı açık), `no_alt_sinir_say` (lower_bound), `soyad_onek_ara` (lower_bound + önek).
- Referans ölçüm: 10^5 kayıtta doğrusal 190,5 µs, ikili 261 ns (**730 kat**).
- Mükerrer denetimi ikili aramaya geçer: karşılaştırma azalır, süre azalmaz (kaydırma, Örnek 8).
- 10 kabul ölçütü ve test senaryoları proje belgesinde.

Atölye: üç etkinlik

Ayrıntılar ders notu Bölüm 11.8–11.10'da; dosyalar `kodlar/alistirmalar/` altında.



1 · Doğru ikili aramayı yaz (bireysel, ≈ 30 dk)

- 20 dk, internet ve not kapalı: `ogrenci_ara`'yı yazın (`H04_A06_test_paketi`).
- Diziye yalnız `eleman(i)` ile erişin; `eleman(mid)` adımda bir kez.
- Paket 4355 sorgu yapar, dört kategori sayar: yanlış sonuç, dizi dışı, sonsuz döngü, çok yoklama.



2 · Sayı tahmin oyunu (eşli, ≈ 20 dk)

- `H04_A07_sayi_tahmin`: sayıyı bilgisayar ya da öğrenci tutar.
- Tur 1: N = 100 ile on oyun; soru sayılarınızı Örnek 10'un dağılımıyla karşılaştırın.
- Tur 2: bilerek bir yanlış yanıt verin; program ne der, neden?



3 · Varyant avı (eşli, ≈ 25 dk)

- `H04_A08_varyant_avi`: beş problemi lower_bound ile O(log n)'de çözün.
- adet · aralıkta kaç eleman · en yakın · eksik sayı · ilk kötü sürüm.
- Kâhin ve yoklama sınırı hazır: yer tutucular beşinde de KALDI verir.

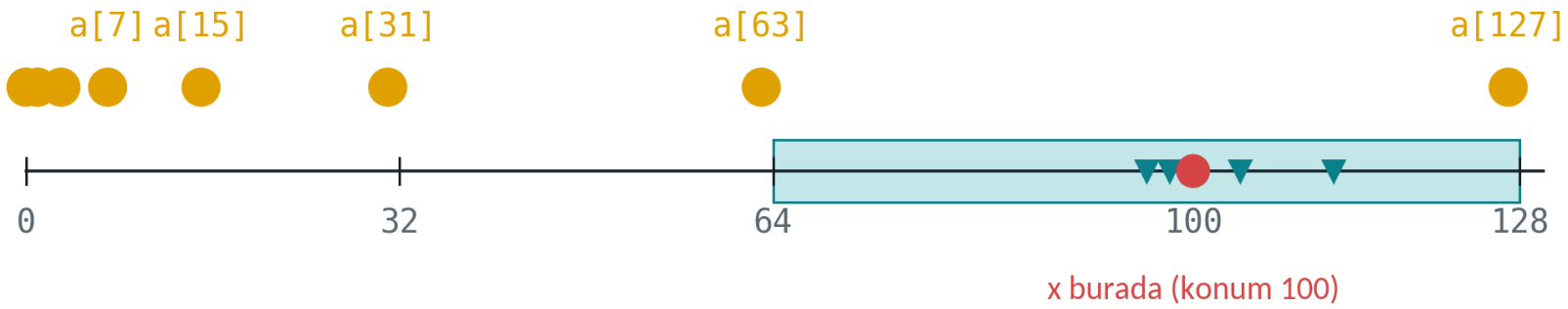
İLERİ BAKIŞ

Bu haftanın konusu nerede derinleşiyor?

- İkili arama ağaçları ve dengeli ağaçlar, Veri Yapıları dersinde sıralı dizinin dinamik karşılığı olarak ele alınır.
- H5: araya ekleme sıralaması = Örnek 8'in sıralı eklemesi; H10: karma tabloları eşitlik aramasını ortalama O(1)'e indirir ama sıra bilgisini kaybeder.
- Cevap üzerinde ikili arama; optimizasyon problemlerinde ve yarışma programlamada sık kullanılır; H13'te en uzun artan alt dizinin hızlı sürümü lower_bound kullanır.
- git bisect: yazılım geçmişinde hatayı ikili aramayla bulma (Mühendis Gözüyle).

Kısa bakış: üstel arama ve interpolasyon araması

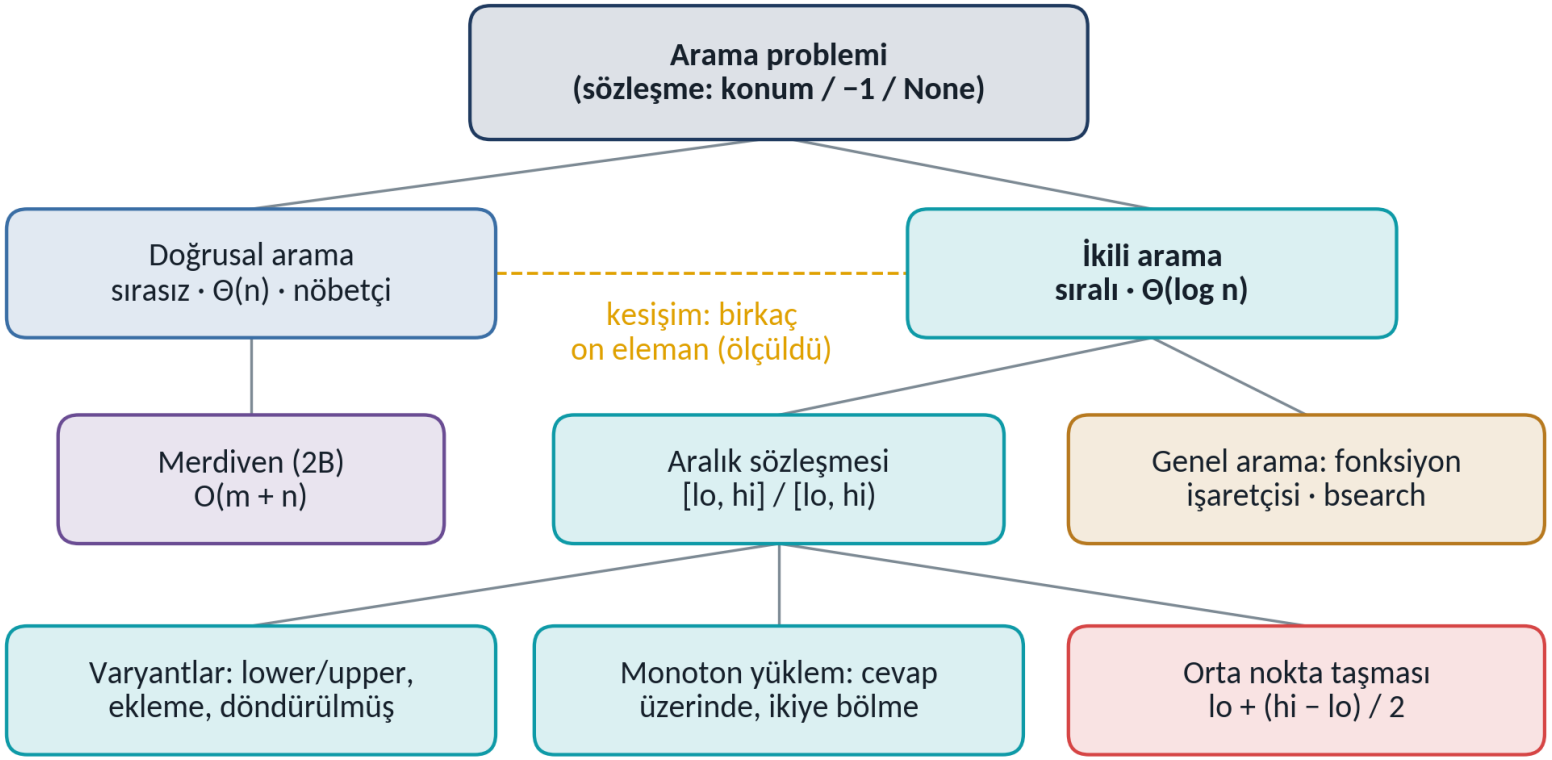
Üstel arama, x = a[100]: sınır 1, 2, 4, … ikiye katlanır (8 yoklama), sonra [64, 128) içinde ikili arama (6 yoklama)



konum k ise yoklama ≈ $2 \cdot \log_2 k$ (n'den bağımsız); bu örnekte 14, n = 10^6'da tüm dizide ikili arama 20

- **Üstel arama:** sınır 1, 2, 4, … ikiye katlanır; sonra yalnızca [sınır/2, sınır) içinde ikili arama.
- Konum k ise O(log k): n = 10^6'da k = 5 → 7, k = 100 → 14 yoklama (ikili 20); k = 999 999 → 39.
- **İnterpolasyon:** mid yerine x'in değerine göre kestirilen konum yoklanır (rehberde “S”yi sonda açmak).
- n = 10^5, ortalama yoklama: düzgün dizi 3,25 (ikili 15,75); a[i] = i^3 dizisi 114,41.

Özet: haftanın kavram haritası



- Sözleşme: bulunamadı, tekrar, ön koşul.
- $\lfloor \log_2 n \rfloor + 1$ yoklama: alt sınır.
- Dört öge uyumlu; mid her adımda çıkar.
- lo + (hi − lo) / 2.
- Monoton yüklemde ilk D.

ÇIKIŞ BİLETİ

Dersten çıkmadan önce üç soru

1. Yarı açık aralıkla ikili aramanın dört ögesini (başlangıç, koşul, iki güncelleme) yazın.

2. [1, 3, 3, 3, 5, 8] dizisinde x = 3 için lower_bound ve upper_bound kaçtır?

3. C'de `(lo + hi) / 2` neden hatalıdır, Python'da neden değildir?

Gelecek haftaya hazırlık: temel sıralama

- **Oku:** ders notu Bölüm 5.2 (sıralı ekleme) ve 8 (karşılaştırıcılar); H5'in Büyük Sorusu.
- **Çalıştır:** `kodlar/alistirmalar/` altındaki atölye dosyaları; Atölye 1'in test paketini GEÇ'e getirin.
- **Dene:** web sayfasındaki ikili arama oynatıcısında kapalı ve yarı açık sözleşmeyi karşılaştırın; varyant karşılaştırıcısında x = 9.
- **Düşün:** n elemanı tek tek sıralı diziye eklemek kaç kaydırma ister? (Örnek 8: ≈ $n^2/4$)
- Mini-OBS 2 v2.3: kabul ölçütleri 1–4'ü gelecek derse kadar tamamlayın.